

Age-Verified User Groups & Investment Contests System

Context

Beanstalk currently has a user model with basic age capture (user_age as DATE type) and a single agreement acceptance flag. To enable age-grouped investment contests where users compete with peers in their age demographic, we need to:

1. **Extend the user model** to capture demographic data and age group classification
2. **Implement age-specific EULA/consent handling** with versioned terms for different age groups
3. **Create contest system** with age-group matching for fair competition
4. **Develop web portal roles** for contest management, moderation, and analytics
5. **Build portfolio tracking** with virtual currency that can be used in contests
6. **Add leaderboards** for age-grouped contest results

This enables the core feature: kids and young adults can participate in age-appropriate investment contests (paper trading) against peers of similar age.

Design Overview

1. USER MODEL EXTENSIONS

New User Fields to Add:

```
age_group: ENUM ('middle_school' | 'high_school' | 'college' | 'adult') // Calculated from DOB
date_of_birth: DATE (replace current user_age)
demographics: JSON/RECORD {
  gender: STRING (optional)
  education_level: ENUM ('middle_school' | 'high_school' | 'college' | 'graduate')
  state_of_residence: STRING
  interests: ARRAY (e.g., ['tech', 'healthcare', 'finance'])
  risk_tolerance: ENUM ('conservative' | 'moderate' | 'aggressive')
}
consent_tracking: RECORD {
  tos_version: STRING (e.g., "v1.2")
  tos_accepted_at: DATETIME
  tos_accepted_age_group: ENUM (age group when they accepted)
  privacy_policy_version: STRING
  privacy_accepted_at: DATETIME
  parental_consent: RECORD (for under 18) {
    parent_email: STRING
    parent_verified: BOOLEAN
    parent_verified_at: DATETIME
    parent_consent_token: STRING
  }
}
```

```

contest_rules_accepted: BOOLEAN
contest_rules_accepted_at: DATETIME
}
virtual_portfolio: RECORD {
portfolio_id: STRING (UUID)
starting_balance: DECIMAL (default $100,000)
current_balance: DECIMAL
created_at: DATETIME
updated_at: DATETIME
}
contest_eligibility: RECORD {
can_create_contests: BOOLEAN (default false for under 18)
can_join_contests: BOOLEAN (default true if age verified)
contest_level: ENUM ('beginner' | 'intermediate' | 'advanced')
}
account_status: ENUM ('pending_verification' | 'active' | 'suspended' | 'deleted')

**File to Modify:**
- `/Users/mehtafam/Downloads/api/src/GoogleCloudPlatform/Models/user/user.model.js`

---
```

2. AGE GROUP SYSTEM

```

**Age Group Classification Logic:**
13-14 years → 'middle_school'
15-18 years → 'high_school'
19-25 years → 'college'
26+ years → 'adult'

**Validation Rules:**
- Minimum age: 13 (pre-teen/early teen access)
- Maximum age: Accept any reasonable age
- Under 18: Require parental consent/verification
- Age calculated from date_of_birth at signup

**File Structure:**
- Create: `/Users/mehtafam/Downloads/api/src/utils/age_group_calculator.js`
- Function: `calculateAgeGroup(dateOfBirth)` → returns age_group enum
- Function: `calculateAge(dateOfBirth)` → returns age in years
- Function: `isOldEnoughForContest(age, contestType)` → returns boolean
- Function: `requiresParentalConsent(age)` → returns boolean

---
```

3. EULA & CONSENT MANAGEMENT

```

**Consent Document Types:**
1. **Master Terms of Service** - For all users
2. **Age-Group-Specific Supplement** - Varies by age group
3. **Investment/Contest Rules** - For contest participation
4. **Privacy Policy** - For all users, consistent
5. **Parental Consent Form** (under 18 only)

**Consent Storage:**
consent_agreements table/collection:
```

agreement_id: STRING
agreement_type: ENUM ('tos' | 'privacy' | 'contest_rules' | 'parental_consent')
agreement_version: STRING (e.g., "v2.1")
age_group_applicable: ARRAY or null (null = all users)
effective_date: DATE
content: TEXT (full agreement text)
created_at: DATETIME

user_consent_acceptance table/collection:

acceptance_id: STRING
user_id: STRING
agreement_id: STRING
accepted_at: DATETIME
ip_address: STRING
user_age_at_acceptance: INTEGER
device_info: JSON

****Files to Create:****

- `/Users/mehtafam/Downloads/api/src/models/consent/consent.model.js` - Schema definitions
- `/Users/mehtafam/Downloads/api/src/controllers/api/consent/consent.controller.js` - CRUD operations
- `/Users/mehtafam/Downloads/api/src/routes/api/consent.route.js` - API routes

****Files to Modify:****

- Mobile onboarding: Add age-group-specific consent display
- Web portal: Add consent management for admins

4. PORTFOLIO & VIRTUAL CURRENCY SYSTEM

****Portfolio Model:****

portfolio: RECORD {
 portfolio_id: STRING (UUID)
 user_id: STRING
 starting_balance: DECIMAL (100000.00 by default)
 current_cash_balance: DECIMAL
 positions: ARRAY {
 position_id: STRING
 symbol: STRING
 quantity: DECIMAL
 purchase_price: DECIMAL
 purchase_date: DATETIME
 current_value: DECIMAL (updated hourly from Alpaca API)
 }
 total_portfolio_value: DECIMAL (sum of cash + all position values)
 total_return_percent: DECIMAL
 created_at: DATETIME
 updated_at: DATETIME
}

portfolio_transaction: RECORD {
 transaction_id: STRING
 portfolio_id: STRING
 user_id: STRING
 type: ENUM ('buy' | 'sell' | 'dividend' | 'contest_entry' | 'contest_prize')
 symbol: STRING (null if transfer)
 quantity: DECIMAL

```

price: DECIMAL
amount: DECIMAL (quantity * price or transfer amount)
transaction_date: DATETIME
settlement_date: DATETIME
contest_id: STRING (if contest-related)
}

**Files to Create:**
- `/Users/mehtafam/Downloads/api/src/models/portfolio/portfolio.model.js`
- `/Users/mehtafam/Downloads/api/src/controllers/api/portfolio/portfolio.controller.js`
- `/Users/mehtafam/Downloads/api/src/routes/api/portfolio.route.js`

---
```

5. CONTEST SYSTEM

```

**Contest Model:**
contest: RECORD {
  contest_id: STRING (UUID)
  creator_id: STRING (admin or contest manager user)
  name: STRING
  description: TEXT
  age_groups: ARRAY (['high_school', 'college']) // which groups can enter
  start_date: DATETIME
  end_date: DATETIME
  starting_balance: DECIMAL (everyone gets same balance)
  rules: TEXT
  max_participants: INTEGER
  current_participants: INTEGER
  status: ENUM ('draft' | 'active' | 'concluded' | 'cancelled')
  created_at: DATETIME
  updated_at: DATETIME
  concluded_at: DATETIME (when contest ended and winners determined)
}

contest_participant: RECORD {
  participation_id: STRING
  contest_id: STRING
  user_id: STRING
  age_group: STRING (snapshot of user's age group at entry)
  portfolio_snapshot_id: STRING (special portfolio for this contest)
  entry_date: DATETIME
  status: ENUM ('active' | 'withdrew' | 'disqualified')
  final_rank: INTEGER
  final_return_percent: DECIMAL
}

contest_leaderboard: RECORD {
  leaderboard_id: STRING
  contest_id: STRING
  age_group: STRING (which age group this leaderboard is for)
  rankings: ARRAY {
    rank: INTEGER
    user_id: STRING
    username: STRING
    portfolio_value: DECIMAL
  }
}
```

```

total_return_percent: DECIMAL
position_count: INTEGER
best_trade: STRING (symbol of best performing position)
}
updated_at: DATETIME (updated hourly)
}

contest_prize: RECORD {
prize_id: STRING
contest_id: STRING
rank_from: INTEGER (1 for 1st place, 2 for 2nd, etc)
rank_to: INTEGER
prize_description: STRING
prize_type: ENUM ('badge' | 'certificate' | 'virtual_currency' | 'real_prize')
prize_value: STRING (e.g., "$50", "Gold Badge", "1000 virtual points")
}

```

****Files to Create:****

```

- `/Users/mehtafam/Downloads/api/src/models/contest/contest.model.js`
- `/Users/mehtafam/Downloads/api/src/controllers/api/contest/contest.controller.js`
- `/Users/mehtafam/Downloads/api/src/routes/api/contest.route.js`

```

6. WEB PORTAL ROLES & USER MANAGEMENT

****New Web Portal User Roles:****

1. ****Admin**** - Manage all platform features

- Verify user ages
- Create/edit/delete contests
- View all user data (with privacy safeguards)
- Manage other admins and staff
- View analytics dashboard

2. ****Contest Manager**** - Create and manage contests

- Create/edit/publish contests
- View contest leaderboards
- Announce contest winners
- Cannot delete contests or access user data

3. ****Moderator**** - Monitor for abuse/fraud

- Flag suspicious accounts
- Review age verification attempts
- Monitor for market manipulation in contests
- Issue account suspensions (for review by admin)

4. ****Analytics Officer**** - View data and metrics

- View aggregated contest metrics
- Export anonymous contest data
- View user metrics (anonymized)
- Generate reports

****File to Modify:****

```

- `/Users/mehtafam/Downloads/web/src/app/core/account/account.types.ts` - Add role definitions
- Create: `/Users/mehtafam/Downloads/api/src/models/web-portal-user/web-portal-user.model.js` - New user type for web portal staff

```

7. API ENDPOINTS

****User Management (Extend existing):****

PUT /api/user/:user_id/demographics

Request: { education_level, state, interests[], risk_tolerance }

Response: { demographics_updated }

PUT /api/user/:user_id/age-group

Auto-calculates on DOB change, read-only endpoint for frontend

GET /api/user/:user_id/age-group

Response: { age_group, date_of_birth, age, requires_parental_consent }

POST /api/user/:user_id/parental-consent/request

Request: { parent_email }

Sends verification link to parent email

POST /api/user/:user_id/parental-consent/verify

Request: { verification_token }

Response: { consent_verified, account_activated }

****Consent Management:****

GET /api/consent/active

Response: { tos, privacy_policy, contest_rules } - Latest versions

POST /api/consent/accept

Request: { agreement_ids: ['tos_v2.1', 'contest_rules_v1.0'] }

Response: { accepted_at, user_consent_record }

GET /api/consent/history/:user_id

Response: { acceptance_history: [{agreement_id, accepted_at, version}] }

****Portfolio Management:****

GET /api/portfolio/:user_id

Response: { portfolio: {starting_balance, current_balance, positions[], total_return} }

POST /api/portfolio/:user_id/trade

Request: { action: 'buy'|'sell', symbol, quantity, price }

Response: { transaction_id, portfolio_updated }

GET /api/portfolio/:user_id/transactions

Response: { transactions: [{type, symbol, quantity, date, amount}] }

****Contest Management:****

POST /api/contests

Request: (admin/manager only) { name, description, age_groups[], start_date, end_date, rules }

Response: { contest_id, status: 'draft' }

GET /api/contests

Response: { contests: [{id, name, age_groups, status, start_date, end_date, participants_count}] }

GET /api/contests/:contest_id/leaderboard/:age_group

Response: { rankings: [{rank, username, portfolio_value, return_percent}] }

POST /api/contests/:contest_id/join

Request: { user_id }

Response: { participation_id, contest_portfolio_id, status }

POST /api/contests/:contest_id/conclude

(admin/manager only) Finalize contest, calculate winners, assign prizes

Response: { concluded_at, ranked_participants, prizes_awarded }

****Web Portal Admin Endpoints:****

GET /api/admin/users

Filters: age_group, account_status, verification_status

Response: { users: [{user_id, username, age_group, created_at, status}] }

PUT /api/admin/users/:user_id/verify-age

Request: { verified: true|false, verification_method: 'self-reported'|'id_verified' }

Response: { verified_at, account_status }

GET /api/admin/analytics/contests

Response: { contest_stats, participation_by_age_group, winner_distribution }

GET /api/admin/analytics/users

Response: { new_users_by_age_group, retention_rates, engagement_metrics }

8. FRONTEND CHANGES

****Mobile App (Flutter):****

- Extend onboarding: Add demographic capture (education, interests, risk tolerance)
- Add step for age verification with parental consent flow
- Display age-group-specific EULA/consent
- Add portfolio dashboard
- Add contest discovery/leaderboard screens
- Implement portfolio trading UI

****Web Portal (Angular):****

- Create Admin Dashboard:
- User management & age verification panel
- Contest creation/management interface
- Leaderboard viewer
- Analytics dashboard
- Implement access control by role
- Consent document management UI
- Moderator tools for flagging suspicious activity

Implementation Phases

Phase 1: Foundation (Backend)

1. Extend user model schema with age_group, demographics, consent_tracking
2. Create age_group_calculator utility
3. Create consent management system (models, controllers, routes)
4. Database migrations for new fields

****Critical Files:****

- `/Users/mehtafam/Downloads/api/src/GoogleCloudPlatform/Models/user/user.model.js`
- `/Users/mehtafam/Downloads/api/src/Utils/age\_group\_calculator.js`
- `/Users/mehtafam/Downloads/api/src/models/consent/consent.model.js`
- `/Users/mehtafam/Downloads/api/src/controllers/api/consent/consent.controller.js`

- `/Users/mehtafam/Downloads/api/src/routes/api/consent.route.js`

Phase 2: Portfolio & Contest Systems (Backend)

1. Implement portfolio model and API
2. Implement contest model and API
3. Create leaderboard calculation logic
4. Integrate with Alpaca trading data for portfolio updates

Phase 3: Web Portal Infrastructure

1. Define web portal user roles and permissions
2. Implement role-based access control
3. Create admin dashboard components
4. Build consent management UI

Phase 4: Mobile App Integration

1. Update onboarding flow with demographics & parental consent
2. Add portfolio dashboard screens
3. Add contest discovery/entry screens
4. Implement leaderboard viewers

Phase 5: Testing & Launch

1. End-to-end testing across mobile, web, backend
2. Age verification security testing
3. Contest fairness/integrity testing
4. Performance testing with real Alpaca data

Key Technical Decisions

1. ****Age Group Calculation:**** Done server-side at signup and stored as enum for consistency
2. ****Virtual Currency:**** Stored as DECIMAL in user's portfolio record, not separate ledger system
3. ****Portfolio Updates:**** Can be updated real-time from Alpaca API or batch processed hourly
4. ****Consent Versions:**** Each agreement type has independent versioning; users must accept latest versions
5. ****Contest Fairness:**** Each participant gets snapshot portfolio for contest duration, separate from main portfolio
6. ****Parental Consent:**** Can be optional in v1 (collect email address), enforce verification in v2

Verification & Testing

1. ****Age Group Classification:****
 - Test `date_of_birth` → `age_group` mapping for each group boundary
 - Verify parental consent required for under 18
2. ****Consent Tracking:****
 - Accept consent → verify stored in `consent_acceptance` table

- Retrieve user consent history → verify all versions tracked
- Age-group-specific consent → verify correct version shown by age

3. **Portfolio & Trading:**

- Create portfolio → verify starting balance correct
- Execute trade → verify transaction recorded and balance updated
- Compare to Alpaca data → verify position values match

4. **Contests:**

- Create contest → verify draft status
- Join contest → verify separate portfolio created
- Conclude contest → verify rankings calculated and prizes assigned
- Check leaderboards → verify age-group filtering works

5. **Web Portal:**

- Login with each role → verify access control enforced
- Admin view users → verify age verification flags visible
- Contest manager create contest → verify contest appears in list